Week 8: Monte Carlo (MC)

MATH-517 Statistical Computation and Visualization

Linda Mhalla

2026-11-06

Introduction

MC $\equiv$ repeated random sampling to mimic outcome of random process and produce numerical results such as

- generating draws from complicated distributions and/or domains
- integration: calculation of moments or high-dimensional densities

Basic reasons to simulate

- **exploration**: Is my idea any use? Should I take it further, or should I give up and try something else?
- **inference**: CIs or hypothesis testing
- **confirmation**: How well does this procedure work? Does it improve on existing methods? When can I trust it?

Basic idea: If we can sample from a process or mimic its outcomes, we can learn a lot about it by doing statistics on the simulated samples (as opposed to analyzing the process itself)

MC methods $\equiv$ simulation-based statistical techniques/inference

Introduction

Gambling experiments have random outcomes – hence "Monte Carlo"



Method initially developed by Stanislaw Ulam and John von Neumann for the Manhattan Project on neutron-transport and related nuclear calculations (multidimensional integrals)

Example

$(X_1, Y_1)^\top, \ldots, (X_N, Y_N)^\top$ a sample from the standardized bivariate Gaussian distribution

$$\mathcal{N}\left(\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1 & \rho\\ \rho & 1\end{pmatrix}\right)$$

- We want to test $H_0 : \rho = \rho_0$ against $H_1 : \rho \neq \rho_0$
- Statistic: $\hat{\rho}_N = N^{-1} \sum_{n=1}^{N} X_n Y_n$

Since the data generation process is fully determined under H_0, we can simulate data to approximate the sampling distribution and thus also the p-value

To test a hypothesis, we only need to simulate data under H_0

But how to draw samples from a specific distribution?

Example: Estimation of π

$$\Pr(\text{point inside the circle}) = \frac{\text{area of the circle}}{\text{area of the square}} = \frac{\pi r^2}{2r \times 2r} = \frac{\pi}{4}$$

```
set.seed(123)
piVal <- function(nPoints = 300){
  x <- runif(nPoints,-1,1)
  y <- runif(nPoints,-1,1)
  result <- ifelse(x^2+y^2 <= 1, TRUE, FALSE)
  4*sum(result)/nPoints
}

N      <- 1000
pi_est <- replicate(N, piVal())
mean(pi_est) #average over samples
```

```
[1] 3.141893
```

Section 1

Random Number Generation (RNG)

RNG

True randomness is hard to come by. Historically:

- dice, cards, coins
- physical processes
- census data, tables, etc.



Practical reasons not to use "truly" random numbers: debugging and reproducibility

John von Neumann: **pseudo-RNG**

- approximates the desired dist. for $N \to \infty$
- cannot be predicted (without the seed or generator)
- pass a set of independence tests
- repeatability ($\Rightarrow$ reproducibility)
- long cycle (before it starts repeating) and fast sampling

Uniformity and independence tests needed to assess quality of pseudo-RNG

Cornerstone: Generating from $\mathcal{U}[0,1]$

Assume now we can generate numbers from the $\mathcal{U}[0,1]$ distribution

- e.g., the *linear congruential method (LCG)*

$$X_n = (aX_{n-1} + c) \bmod m, \quad n = 1, 2, \dots,$$

  where a, c, m, and X_0 are cleverly chosen to fulfill the pseudo-RNG requirements, i.e., maximize period, speed, and "randomness"
  - X_0 is the seed
  - produces integers between 0 and $m-1$
  - $U_n = X_n/m \dot{\sim} U(0,1)$

Bad example: $m = 2^{31}$, $a = 2^{16} + 3$, and $c = 0 \Rightarrow$ IBM's RANDU

Example: $a = 13$, $c = 0$, and $m = 64$

- $X_0 = 4$: 4, 52, 36, 20, 4
- $X_0 = 2$: 2, 26, 18, 42, 34, 58, 50, 10, 2

Choice of parameters

Case	Parameters	Max period	Conditions
Mixed LCG	$m = 2^b,\ c \neq 0$	$P = 2^b$	c relatively prime to m; $a = 1 + 4k,\ k \in \mathbb{Z}$
Multiplicative LCG	$m = 2^b,\ c = 0$	$P = 2^{b-2}$	Seed X_0 odd; $a = 3 + 8k$ or $a = 5 + 8k,\ k = 0, 1, \ldots$
Multiplicative (prime modulus)	m prime, $c = 0$	$P = m - 1$	a must be a *primitive root* mod m (smallest k with $a^k \equiv 1 \pmod{m}$ is $k = m - 1$)

Now, better and much more complicated algorithms are available

- every piece of software has its favorite pseudo-RNG
- outside the scope of the course (see, e.g., shift-register generators or Wichmann–Hill generator)

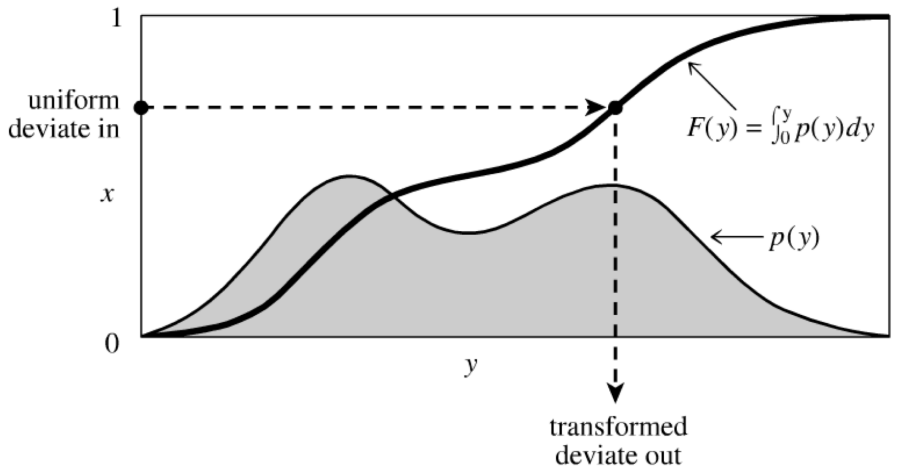
Series not truly random so need to test for randomness (uniformity + independence)

Question: How do we generate from other distributions?

Transforms

Lemma. (*Inverse Transform.*)
Let $U \sim \mathcal{U}(0,1)$ and F be a distribution function and F^{-1} the quantile (or generalized inverse) function. Then $X = F^{-1}(U) \sim F$.

Proof: Simply $P\{F^{-1}(U) \leq x\} = P\{U \leq F(x)\} = F(x)$.



Transforms

The inverse transform method is general, but requires a known and easily invertible CDF, which many important distributions don't have in closed form

- e.g., $\mathcal{N}(0, 1)$

Often, simpler relationships can be used: diagram

- still, there is no arrow there between $\mathcal{U}(0, 1)$ and $\mathcal{N}(0, 1)$, generating $\mathcal{N}(0, 1)$ is actually a bit tricky...

Transforms

Lemma. (*Box–Muller transform.*)
Let $U_1, U_2 \sim \mathcal{U}(0,1)$ be independent. Then

$$Z_1 = \sqrt{-2\log(U_1)}\cos(2\pi U_2) \quad \& \quad Z_2 = \sqrt{-2\log(U_1)}\sin(2\pi U_2)$$

are two independent standard Gaussian random variables

Explanation:

- points from the $\mathcal{N}(0, I_2)$, when written in polar coordinates, have an angle $\theta = \tan^{-1}(Z_1/Z_2) \sim U[0, 2\pi)$ which is independent of the radius R
- the squared radius $R^2 = Z_1^2 + Z_2^2 \sim \chi^2_2$, i.e. $Exp(1/2)$

Again, software uses its favorite relationships

- e.g., `R` has tabulated F and F^{-1} for $\mathcal{N}(0,1)$ to a high precision and actually uses the inverse transform, because evaluating trigonometric functions is rather expensive (slow). `?rnorm` $\Rightarrow$ `rnorm`, `pnorm`, `qnorm`, `dnorm` help
- NumPy uses the Ziggurat algorithm (faster than the others)

Rejection Sampling

Setup: we know how to simulate from a *proposal* g, we want to simulate from a *target* f

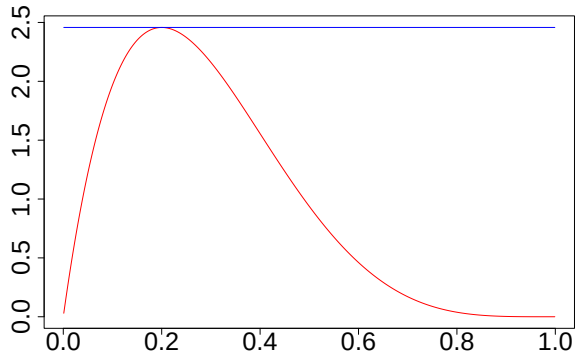
- let $supp(f) \subset supp(g)$, i.e., $f(x) > 0 \Rightarrow g(x) > 0$
- let there be $c > 1$ such that $\forall x : \; f(x) \leq c\, g(x)$, i.e., $\sup_x \frac{f(x)}{g(x)} = c < \infty$

Algorithm: (to obtain a single draw X from f)

1. Draw a proposal Y from g
2. Draw $U \sim \mathcal{U}(0, 1)$
3. If $U \leq \frac{1}{c} \frac{f(Y)}{g(Y)}$, accept $X = Y$ and stop, otherwise go back to 1

Example:

- $\mathcal{U}(0, 1)$ proposal
- $\mathcal{B}(2, 5)$ target
- $c \approx 2.5$



Rejection Sampling

Does the algorithm really sample from f?

$$P(X \leq x) = P\left\{ Y \leq x \,\middle|\, U \leq \underbrace{\frac{1}{c}\frac{f(Y)}{g(Y)}}_{=:t(Y)} \right\} = \frac{P\{Y \leq x,\ U \leq t(Y)\}}{P\{U \leq t(Y)\}}$$

$$= \frac{\int_{-\infty}^{x} \int_{0}^{t(y)} du\, g(y) dy}{\int_{-\infty}^{+\infty} \int_{0}^{t(y)} du\, g(y) dy} = \frac{\int_{-\infty}^{x} t(y) g(y) dy}{\int_{-\infty}^{+\infty} t(y) g(y) dy} = \frac{\int_{-\infty}^{x} \frac{1}{c} f(y) dy}{\int_{-\infty}^{+\infty} \frac{1}{c} f(y) dy}$$

$$= \frac{\frac{1}{c} F(x)}{\frac{1}{c}} = F(x)$$

The rejection sampling algorithm above is again quite general, but it needs

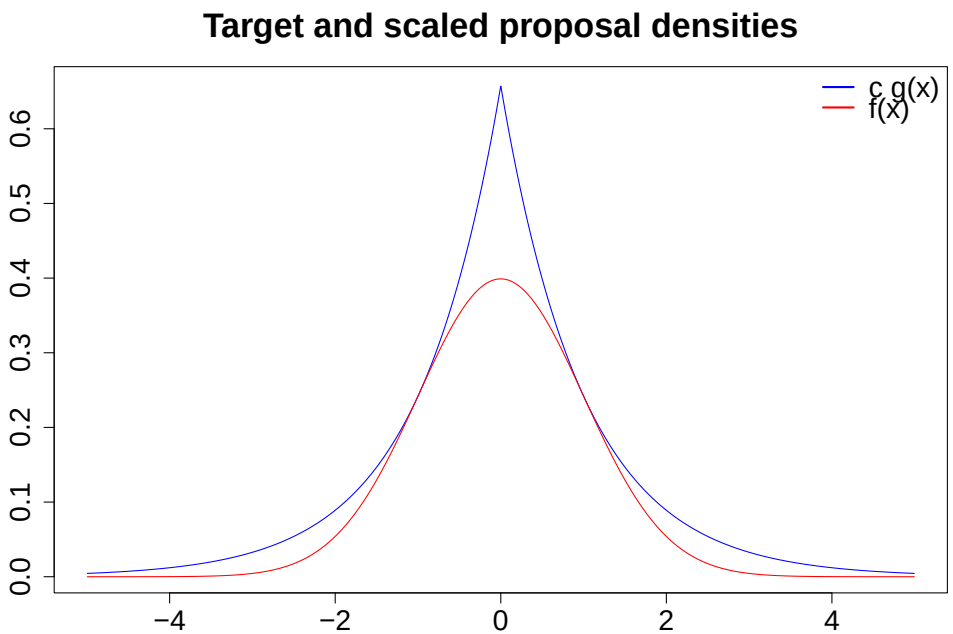
- a good proposal g
  - as close as possible to the target density (small values of c), leading to
  - high acceptance probability $P\{U \leq t(Y)\} = 1/c$
- fast evaluation of f and g

Note: Adaptive rejection sampling for $\log(f)$ concave

Example: $\mathcal{N}(0,1)$ again

Goal: Simulate data from the standard Gaussian target using the double exponential (Laplace) proposal, i.e.,

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} \quad \& \quad g(x) = \frac{\alpha}{2} e^{-\alpha|x|}, \text{ where } \alpha > 0, \quad x \in \mathbb{R}$$



Another way of obtaining $\mathcal{N}(0,1)$ from $\mathcal{U}(0,1)$:

$$\mathcal{U}(0,1) \longrightarrow Exp(1)$$
$$Exp(1) \longrightarrow DbExp(1)$$
$$DbExp(1) \longrightarrow N(0,1)$$

Note: $\alpha = 1$ minimizes the value of $c = \sup f(x)/g(x)$ and hence maximises the acceptance probability

Summary

- Random number generation is an evolving subject: algorithms are continually improved as hardware and software develop
- We discussed how to generate pseudo-random numbers, starting from $U(0, 1)$:
  - linear congruential generators (LCG)
  - inverse transform sampling
  - the Box–Muller transform
  - rejection sampling (mention to adaptive)
- A good pseudo-RNG should have a long period, be reproducible, and behave approximately as independent uniform draws
  - its quality can be assessed using uniformity and independence tests
- In practice, software such as R already provides efficient implementations for generating from many common distributions

Section 2

Numerical Integration

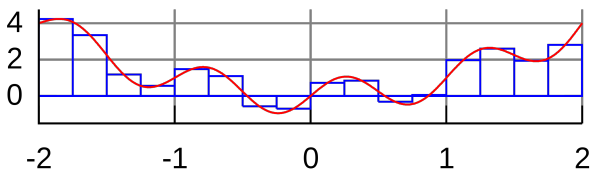
Deterministic Quadrature Approaches

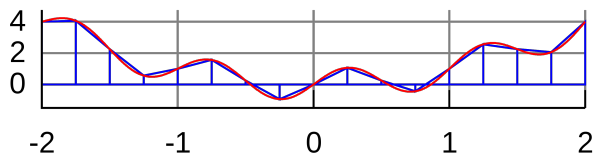
Goal: approximate $J = \int_a^b f(x)dx$ via composite quadrature rules

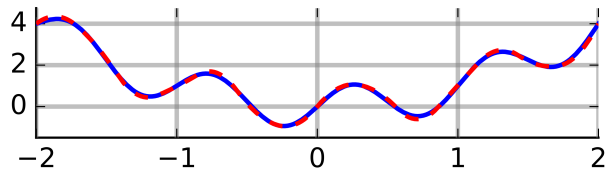
Quadrature method: evaluate the function on a grid

$$S_K = \{(b-a)/K\} \sum_{k=1}^{K} f(t_k)$$

- if f is nice (smooth), $S_K \to J$ for $K \to \infty$
- corresponds to integrating a local constant interpolation of f
- local linear interpolation (*trapezoidal rule*) or local quadratic (*Simpson's rule*) are also well known







Naive Monte Carlo

We consider the more general integral

$$J = \int_{\mathcal{X}} m(x) f(x) dx = \mathbb{E}_f\{m(X)\} \quad \text{for} \quad X \sim f$$

$\Rightarrow$ generate $X_1, \ldots, X_N \overset{i.i.d.}{\sim} f$ and approximate J by

$$\widehat{\bar{J}}_N = N^{-1} \sum_{n=1}^{N} m(X_n)$$

- Provided $\int |m(x)| f(x) dx < \infty$, unbiased and consistent due to SLLN
- monitoring convergence via CLT-based confidence intervals:

$$\sqrt{N} \frac{\widehat{\bar{J}}_N - J}{\sigma(m)} \dot{\sim} \mathcal{N}(0,1), \quad \text{where} \quad \sigma^2(m) = \int m^2(x) f(x) dx - J^2$$

provided $E_f\{m(X)^2\} < \infty$ (naive Monte Carlo estimate v_N^2 of $\sigma^2(m)$ is used in practice)

$\Rightarrow$ MC integration methods give a probabilistic error bound versus a deterministic one with numerical quadrature rules

Naive Monte Carlo

$$\lim_{N\to\infty} \Pr\left(-3\frac{\sigma(m)}{\sqrt{N}} \leq \frac{1}{N}\sum_{n=1}^{N} m\left(X_n\right) - J \leq 3\frac{\sigma(m)}{\sqrt{N}}\right) \approx 99.73\%$$

$\Rightarrow$ to attain an error of $\epsilon > 0$ with 99.73%, we need

$$N = \frac{9}{\epsilon^2}\sigma^2(m) = O(1/\epsilon^2)$$

- This is independent of the dimension (depends on $\mathrm{Var}_f\{m(X)\}$ though)!
- Deterministic numerical integration methods suffer the curse of dimensionality: $N = O(1/\epsilon^d)$, roughly
- Monte Carlo estimate for the integral converges slowly to the true value (slower than quadrature methods for smooth functions in low dimensions)
- Beware of **rare events**: if f has heavy tails, $\widehat{\bar{J}}_N$ can be a bad estimate and we need huge N to get small $v_N \Rightarrow$ techniques for variance reduction

Importance Sampling

We often cannot simulate directly from f and we require sophisticated approaches. Rewrite

$$J := \int_{\mathcal{X}} m(x)f(x)dx = \int_{\mathcal{X}} m(x)\frac{f(x)}{g(x)}g(x)dx = \int_{\mathcal{X}} m(x)w(x)g(x)dx$$

with g a density such that $g(x) > 0$ wherever $m(x)f(x) \neq 0$ and $w(x) \geq 0$ the **importance weighting function**

Thus, by sampling $X_1, \ldots, X_N \overset{\text{i.i.d.}}{\sim} g$, we can approximate J by

$$\widehat{J}_N := N^{-1}\sum_{n=1}^{N} m(X_n)w(X_n)$$

Idea:

- Sample from a convenient proposal g
- Ensure g covers all regions contributing to the integral
- Reweight draws by $w(x) = f(x)/g(x)$

Importance Sampling: Intuitive Explanation

Key: integrating f amounts to integrating f/g under sampling from g

$$J = \int_{\mathcal{X}} m(x)f(x)dx = \mathbb{E}_g\{m(X)w(X)\}$$

- when f is flat (all regions are equally important), use either the naive MC (with uniform sample) or deterministic approaches that need only small samples
- when f is not flat, using a "good" g allows us to encode which regions are important $\Rightarrow$ "importance sampling" (vs rejection sampling)

Of course, it is not always easy to find a "good" g which

- has a similar shape than f and
- from which we can easily sample

As we will see, when $\mathcal{X} = \mathbb{R}$, it is important to match the decay of the tails between the target and reference measures

Importance Sampling: Properties

- unbiased and the variance is given by

$$\begin{aligned}\mathrm{var}(\widehat{J}_N) &= \frac{1}{N}\left\{\int_{\mathcal{X}} m^2(x)\frac{f(x)}{g(x)}f(x)dx - J^2\right\} \\ &= N^{-1}\int_{\mathcal{X}}\left\{m(x)\frac{f(x)}{g(x)} - J\right\}m(x)f(x)dx,\end{aligned}$$

  which is small if $g(x) \approx m(x)f(x)J^{-1}$ or $g(x) \propto m(x)f(x)$

$\Rightarrow$ good choices of g can yield huge improvements in efficiency, e.g.,

- g with similar shape than mf
- maximum principle ($g(x)$ and $m(x)f(x)$ take their maximum at the same value)
- g from the same distribution family as f

For instance, reduction in variance is

$$\mathrm{var}(\widehat{\bar{J}}_N) - \mathrm{var}(\widehat{J}_N) = \frac{1}{N}\int_{\mathcal{X}} m^2(x)\left\{1 - \frac{f(x)}{g(x)}\right\}f(x)dx$$

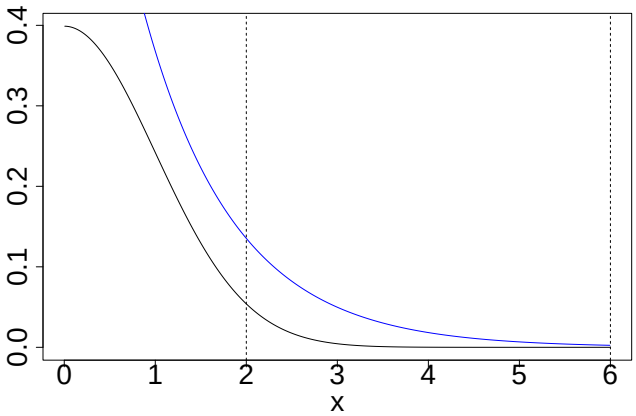
Importance Sampling: Properties

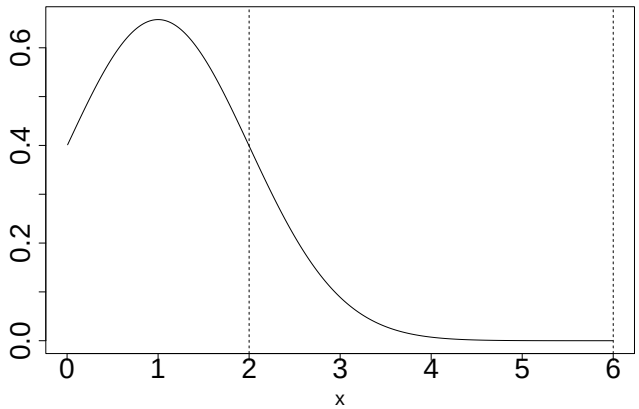
- Most difficult aspect to importance sampling is in choosing a good sampling density g
- Need to be very careful as it is possible to choose g according to some heuristics, but that results in a variance increase
- It is possible to have an importance sampling estimator with infinite variance
  - e.g., if g puts too little weight relative to f on the tails of the distribution (or relative to $|m|f$)

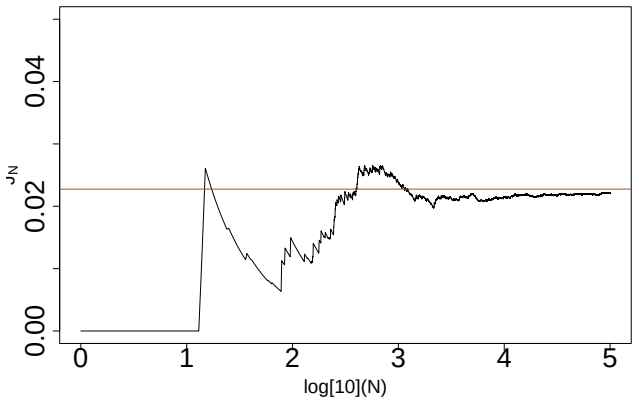
Examples

Task: Approximately calculate $P(2 < X < 6)$ for the target distribution $X \sim f$ using a reference g

Gaussian target, Exponential reference - densities (left), their ratio (middle), importance sampling error (right)



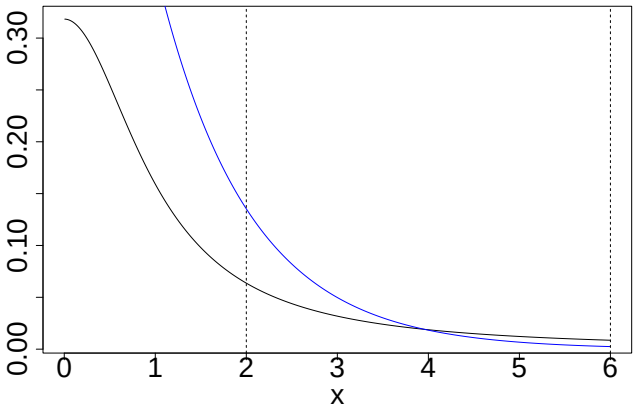


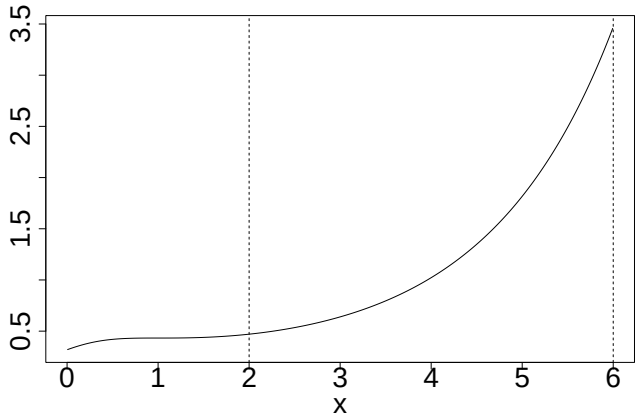


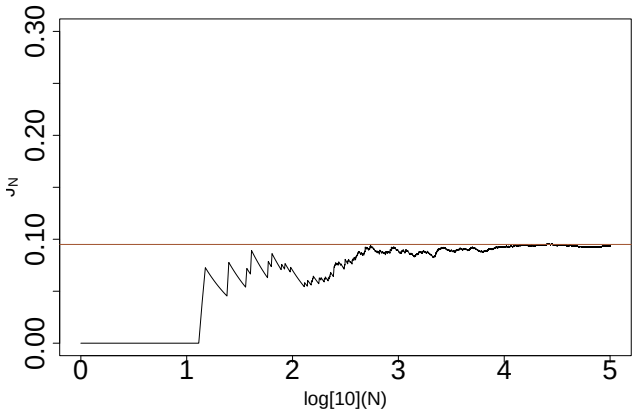
Examples

Approximately calculate $P(2 < X < 6)$ for the target distribution $X \sim f$ using a reference g

Cauchy target, Exponential reference - densities (left), their ratio (middle), importance sampling error (right)



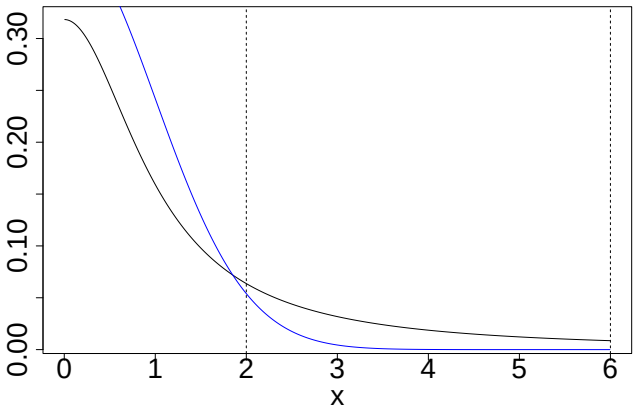


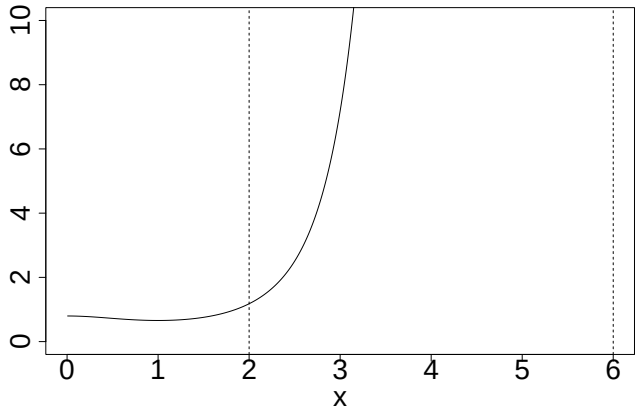


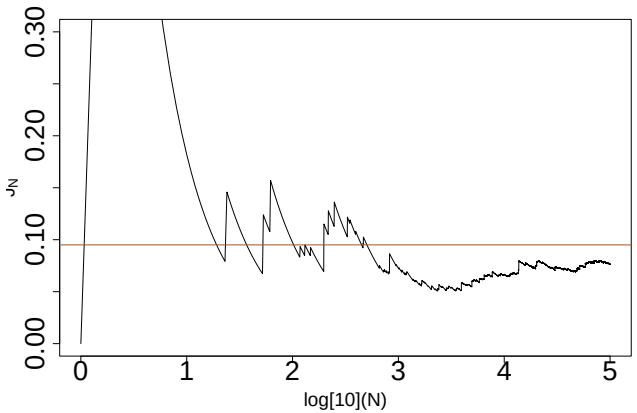
Examples

Approximately calculate $P(2 < X < 6)$ for the target distribution $X \sim f$ using a reference g

Cauchy target, Gaussian reference - densities (left), their ratio (middle), importance sampling error (right)







- the tails of Cauchy and Gaussian distributions are too different (the Gaussian proposal allocates too little mass to parts of the interval that contribute to the probability) $\Rightarrow$ importance sampling performs poorly
- if we can simulate from Gaussian, we can simulate directly from Cauchy: $Z_1, Z_2 \sim \mathcal{N}(0,1)$ independent $\Rightarrow Z_1/Z_2 \sim Cauchy(0,1)$

Variance Reduction

Accuracy of MC integration is assessed by the estimator's efficiency/variance (assuming efforts of simulation are similar)

There are ways to tweak the sampling scheme in order to reduce the variance

- importance sampling (we have seen above)
- antithetic variables (to follow)
- stratified sampling (to follow)
- quasi-random sampling and control variates (see the supplementary notes)
- many other techniques: latin hypercube sampling, ratio estimator, etc

Remark: When comparing several different estimators via simulations, the same simulated datasets should be used for all the estimators

Variance Reduction: Antithetic Variables

Idea: Introduce negative correlation between pairs of replications and rely on

$$\text{var}(f_1 + f_2) = \text{var}(f_1) + \text{var}(f_2) + 2\text{cov}(f_1, f_2)$$

Given two i.d. samples $X_1, \ldots, X_N \sim f$ and $Y_1, \ldots, Y_N \sim f$, consider the estimator

$$\tilde{J}_N = \frac{1}{2N} \sum_{n=1}^{N} \{m(X_n) + m(Y_n)\} = \frac{1}{2}(\hat{\bar{J}}_N^X + \hat{\bar{J}}_N^Y)$$

Then,

$$\text{var}(\tilde{J}_N) = \frac{1}{2}\text{var}(\hat{\bar{J}}_N)\{1 + \text{corr}(\hat{\bar{\text{J}}}_\text{N}^\text{X}, \hat{\bar{\text{J}}}_\text{N}^\text{Y})\}$$

$\Rightarrow \tilde{J}_N$ is more efficient than the naive MC (with sample of size $2N$) if $m(X_n)$ and $m(Y_n)$ are **negatively correlated**

Basic result: if $g(u)$ is monotonic on $0 < u < 1$, then

$$\text{corr}\{g(U), g(1-U)\} < 0$$

Thus $F^{-1}(U)$ and $F^{-1}(1-U)$ are negatively correlated with distribution F

Variance Reduction: Stratified Sampling

- Break sampling space into strata $\Delta_1, \ldots, \Delta_k$ and sample appropriate number of observations in each
- Compute the naive MC estimator in each stratum and sum over all strata
- Beneficial when the strata explain substantial variation in $m(X)$, so that $m(X)$ is relatively homogeneous within each stratum

We want to estimate $J = \mathbb{E}\{m(X)\}$. Let W be another r.v. s.t.

- $p_i = \Pr(W \in \Delta_i)$, with $\Delta_i \subset \mathbb{R}$, is easily computed
- we know how to generate $X \mid W \in \Delta_i$

Assuming $\Delta_1, \ldots, \Delta_k$ is a partition of $\mathbb{R}$ and denoting $J_j = \mathbb{E}\{m(X) \mid W \in \Delta_j\}$ and $\sigma_j^2 = \mathrm{var}\{m(X) \mid W \in \Delta_j\}$, we have

$$J = \mathbb{E}[\mathbb{E}\{m(X) \mid W\}] = \sum_{j=1}^{k} p_j J_j, \quad \text{and } \widehat{J}_{str,N} = \sum_{j=1}^{k} p_j \widehat{\bar{J}}_{j,N_j}$$

Variance Reduction: Stratified Sampling

- Method relies on conditional variance:

$$\text{var}\{m(X)\} = \mathbb{E}[\text{var}\{m(X)|W\}] + \text{var}[\mathbb{E}\{m(X) \mid W\}]$$

  Thus,

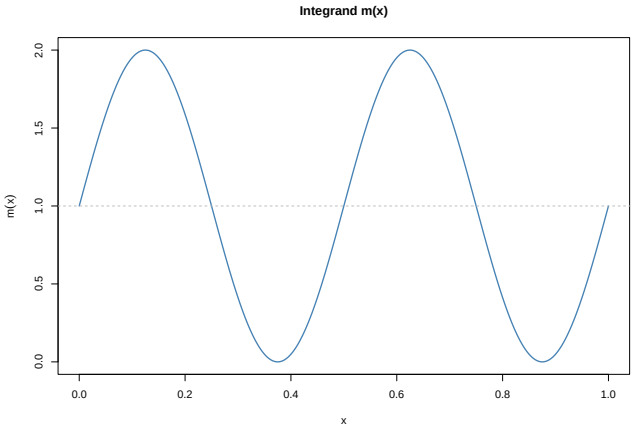
$$\text{var}\{m(X)\} \geq \mathbb{E}[\text{var}\{m(X)|W\}] = \sum_{j=1}^{k} p_j \sigma_j^2$$

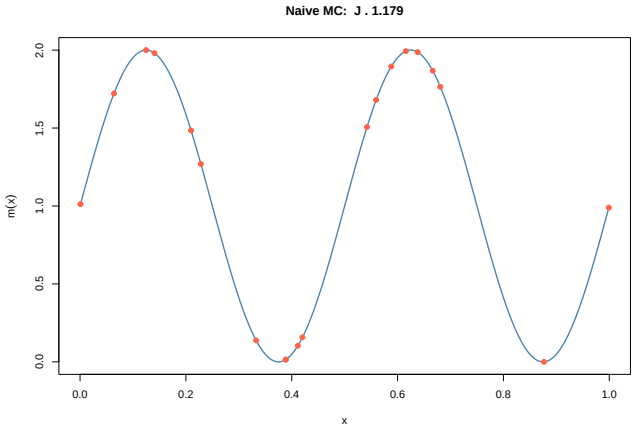
For instance, assuming proportional allocation, i.e., $N_j = Np_j$, then

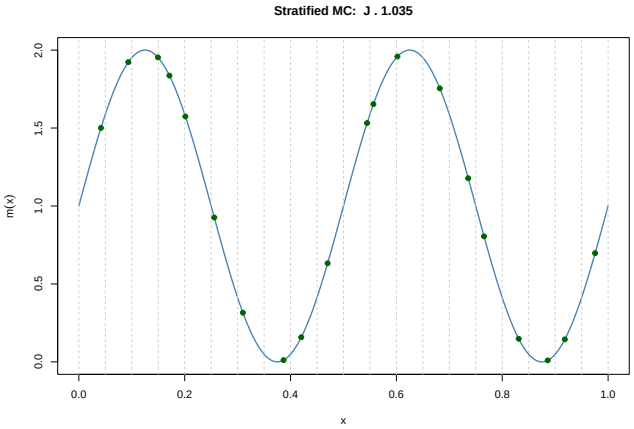
$$\text{var}(\widehat{J}_{str,N}) = \sum_{j=1}^{k} p_j^2 \frac{\sigma_j^2}{N_j} = \frac{\sum_{j=1}^{k} p_j \sigma_j^2}{N} \leq \text{var}(\widehat{\bar{J}}_N) = \frac{\text{var}\{m(X)\}}{N}$$

- Variance reduction substantial if W accounts for a large fraction of the variance of $m(X)$
- Variance reduction depends on the allocation in each stratum. Thus, we can minimize $\text{var}(\widehat{J}_{str,N})$ subject to $\sum_{j=1}^{k} N_j = N \Rightarrow$ more samples go to strata with high variability $N_j \propto p_j \sigma_j$

Variance Reduction: Stratified Sampling







Naive MC can accidentally put many points in some regions and very few in others. Stratified MC forces one point into each interval, giving much more even coverage of the integrand

References

Donald Knuth (1997, 3rd ed.) *The Art of Computer Programming*, vol. 2

Robert & Casella (2010) *Introducing Monte Carlo methods with R*

Assignment

Go to Assignment 1 for details.